

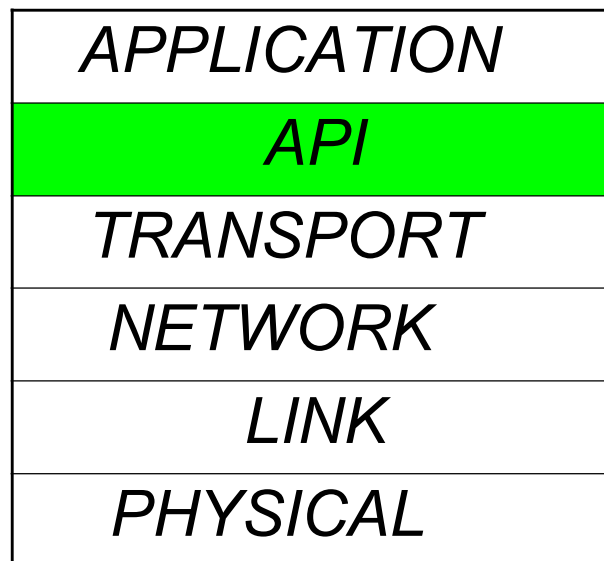
Chapter 5

Data Communications and Networking

Application Layer
Network Programming

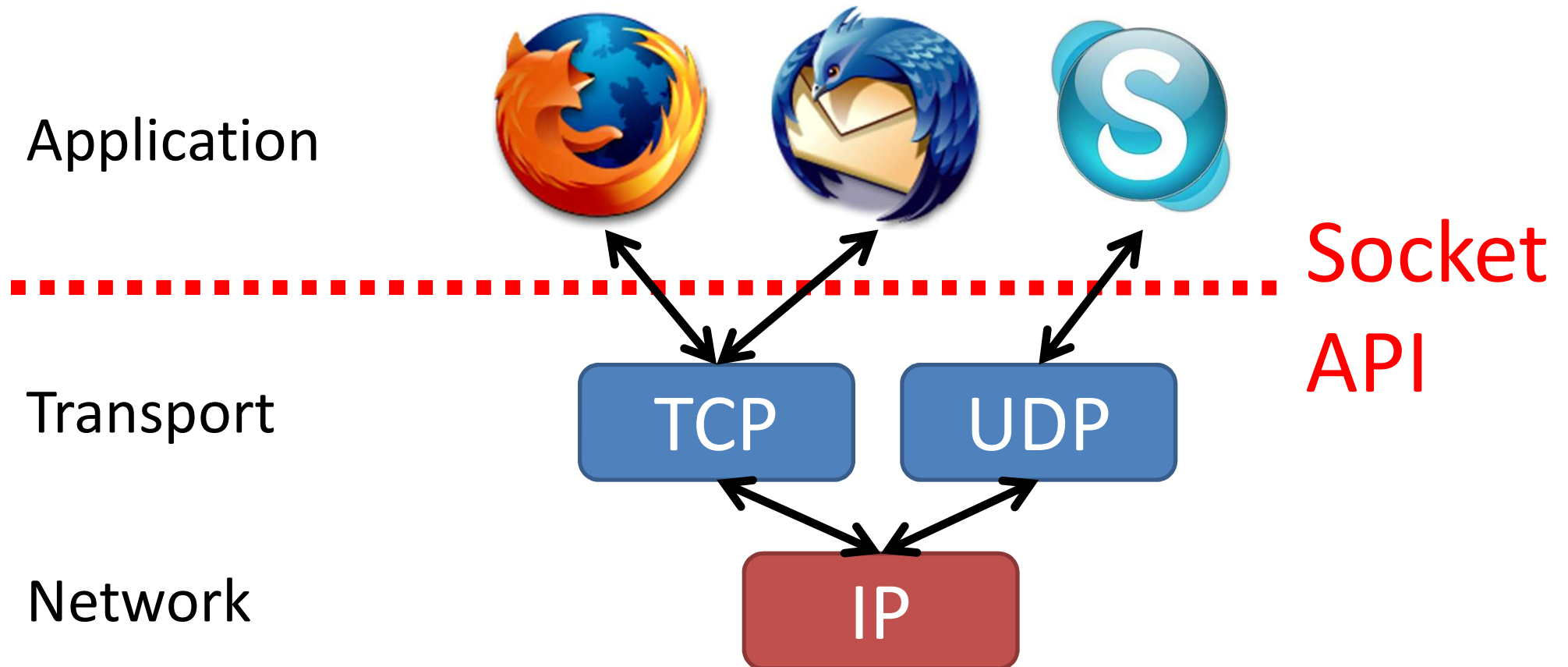
API

- API – stands for Application Programming Interface.
 - Interface to a particular “service”
 - Abstracts away from implementation detail
 - Set of functions, data structures, and constants.
- In our case, it is an interface to use the network.
- A connection to the transport layer.
- Functions at transport layer and below very complex. E.g., to deal with errors on the wireless link and signals to be sent on the network.



Socket API

- Socket API: Network programming interface

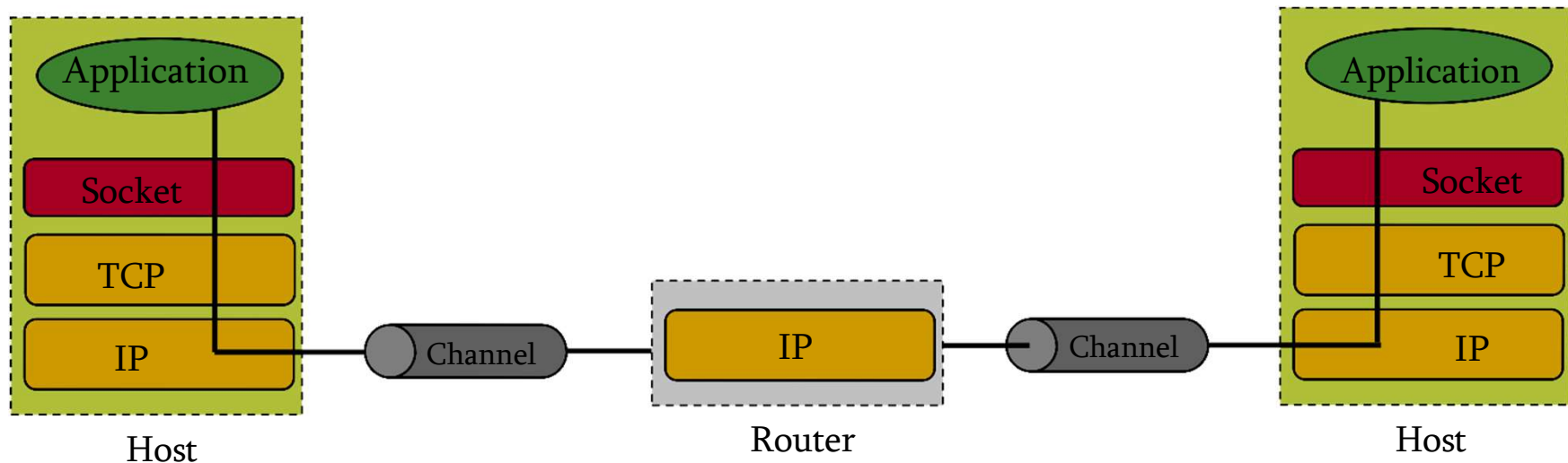


Sockets

- It is an abstraction that is provided to an application programmer to send or receive data to another process.
- Data can be sent to or received from another process running on the same machine or a different machine.
- It is like an endpoint of a connection
- Exists on either side of connection
- Identified by IP Address and Port number
- E.g. Berkeley Sockets in C
 - Released in 1983
 - Similar implementations in other languages

Sockets

- It is an abstraction through which an application may send and receive data
- Provide generic access to interprocess communication services, e.g., IPX/SPX, Appletalk, TCP/IP
- Standard API for networking

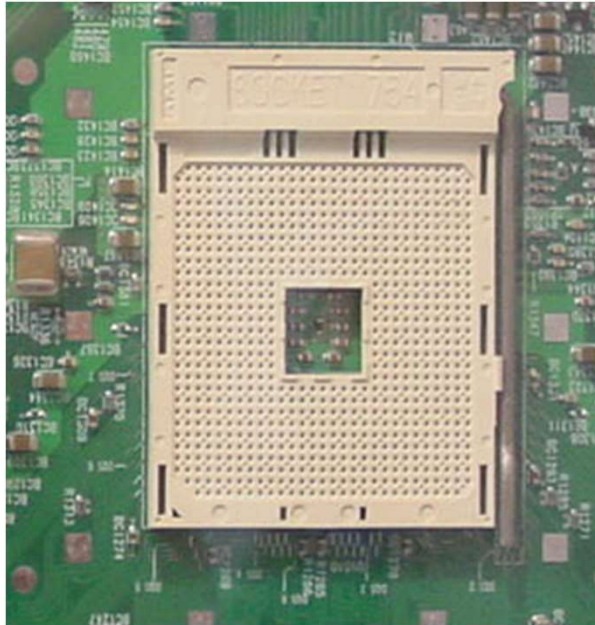


Sockets

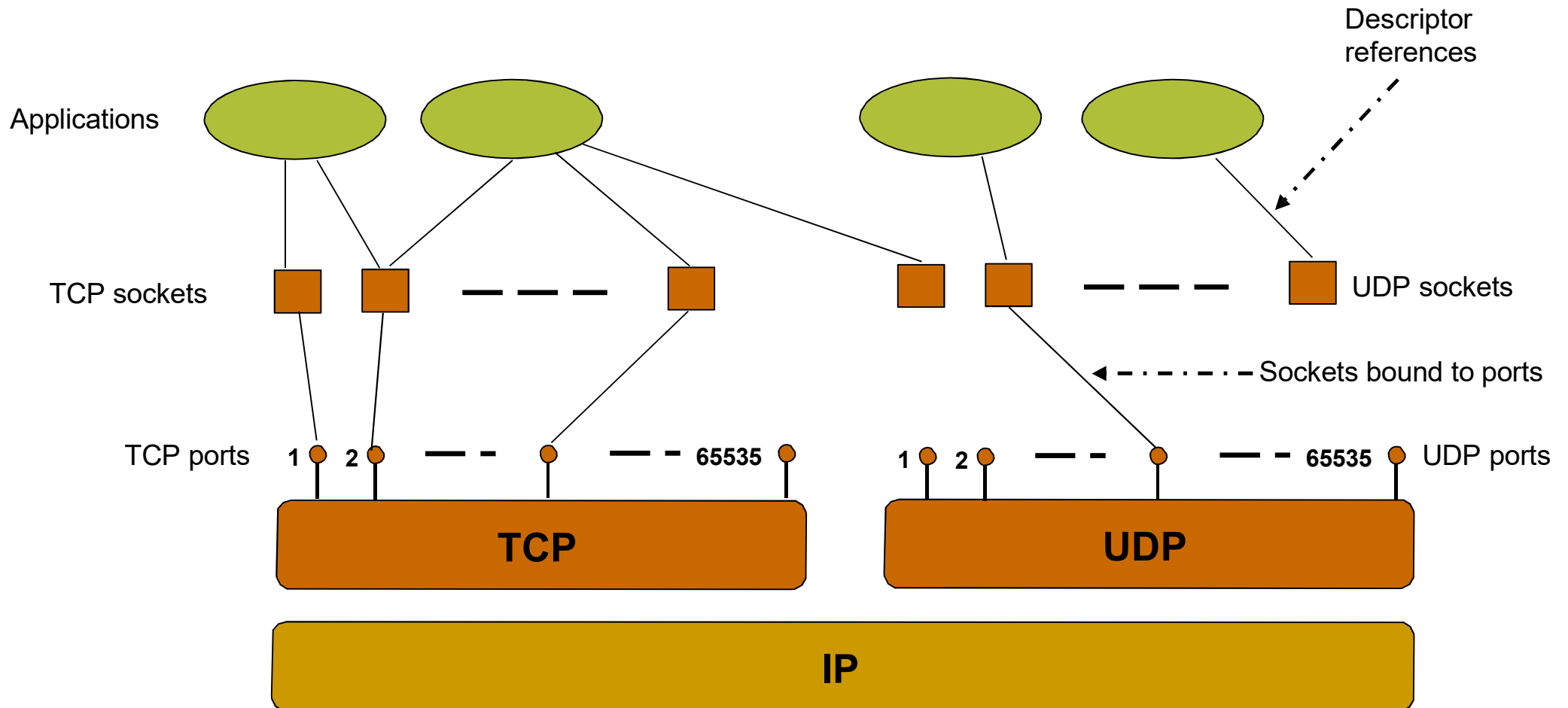
- Uniquely identified by
 - an internet address
 - an end-to-end protocol (e.g. TCP or UDP)
 - a port number
- Two types of (TCP/IP) sockets
 - Stream sockets (e.g. uses TCP): provide reliable byte-stream service
 - Datagram sockets (e.g. uses UDP)
 - provide best-effort datagram service
 - messages up to 65,500 bytes

Sockets

- Similar to various sockets in our daily life



Sockets



Sockets - Procedures

Primitive	Meaning
Socket	Create a new communication endpoint
Bind	Attach a local address to a socket
Listen	Announce willingness to accept connections
Accept	Block caller until a connection request arrives
Connect	Actively attempt to establish a connection
Send	Send some data over the connection
Receive	Receive some data over the connection
Close	Release the connection

Client-Server Communication

- Server
 - passively waits for and responds to clients
 - passive socket
- Client
 - initiates the communication
 - must know the address and the port of the server
 - active socket

Basic Function Calls for Client-Server Communication (TCP, Windows platform)

CLIENT SIDE

WSAStartup

↓
socket

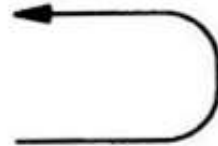
↓
connect

↓
send

↓
recv

↓
closesocket

↓
WSACleanup



SERVER SIDE

WSAStartup

↓
socket

↓
bind

↓
listen

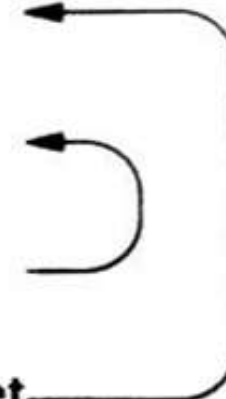
↓
accept

↓
recv

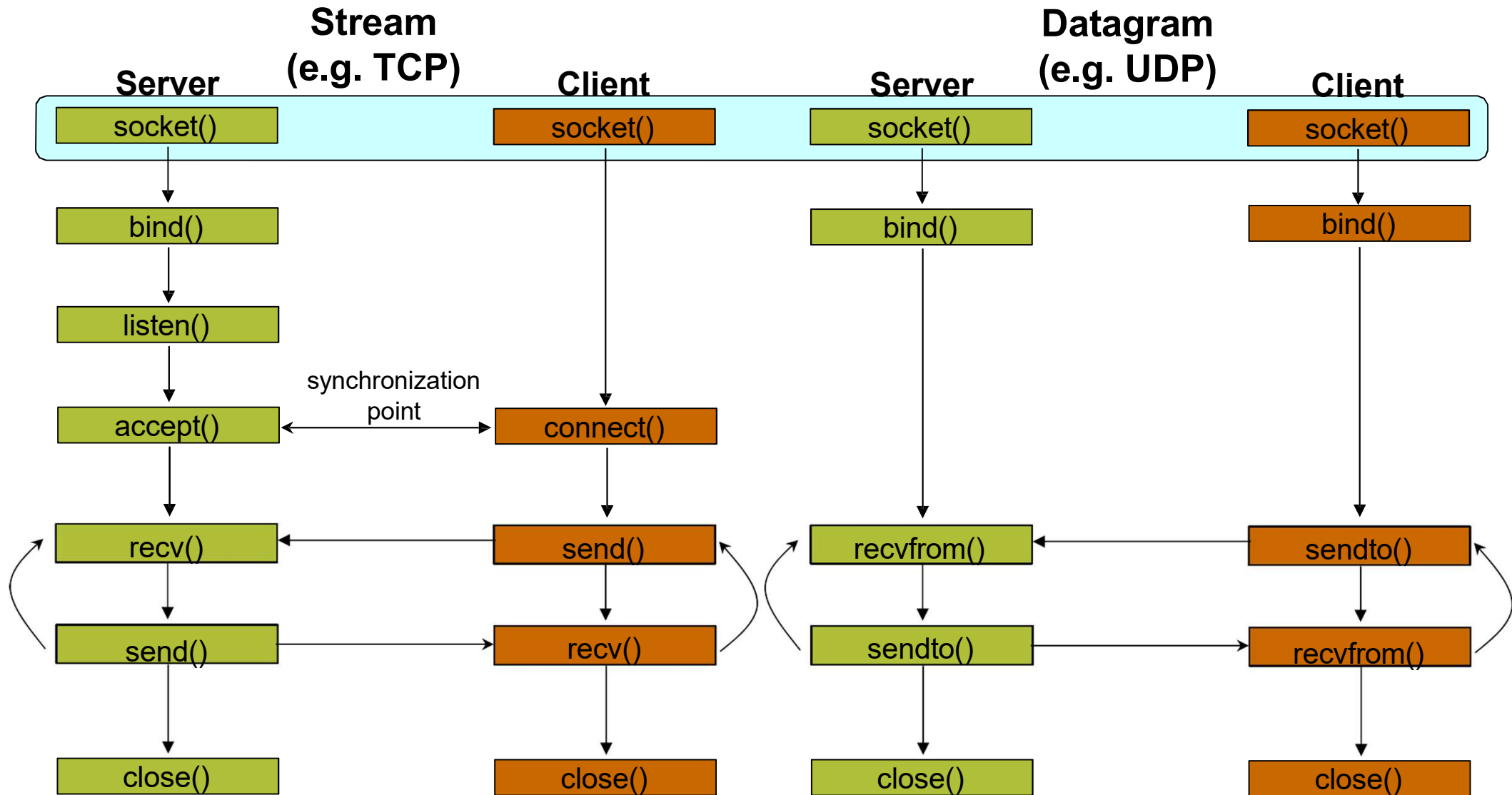
↓
send

↓
closesocket

↓
WSACleanup



Client-Server Communication (Unix platform)



Socket Programming

Introduction to Basic Function Calls

Introduction to Basic Function Calls

- WSAStartup Function
 - Program using Windows Sockets must call WSStartup before using sockets. The call requires two arguments.
 - The program uses the first to specify the version of Windows Sockets that is requested; the OS uses the second to return information about the version of Windows Sockets actually used.

- Syntax:

```
int WSAStartup(WORD wVersionRequested, LPWSADATA lpWSADATA);
```

- Example:

```
WSADATA wsaData;
```

```
if (WSAStartup(0x202, &wsaData) == SOCKET_ERROR){  
    // stderr: standard error are printed to the screen.  
    fprintf(stderr, "WSAStartup failed with error %d\n", WSAGetLastError());  
    //WSACleanup function terminates use of the Windows  
    Sockets DLL. WSACleanup();  
    return -1;  
}
```

Introduction to Basic Function Calls

- WSACleanup Function
 - Once an application finishes using and closing sockets, the application calls WSACleanup to de-allocate all data structures and socket bindings.
- Syntax:
 int WSACleanup(void);
- Example
 closesocket(msg_sock);
 WSACleanup();

Introduction to Basic Function Calls

- Creating a socket
 - To create a socket, you can use the `socket()` function

- Syntax:

```
int socket(int af, int type, int protocol);
```

- Example:

```
int socket_desc;
```

```
socket_desc=socket(AF_INET,SOCK_STREAM,0);
```

```
if (socket_desc==-1)
```

```
perror("Create socket");
```

- `socket()` returns a socket descriptor which can be used in other network commands. If successful, `socket()` returns a valid socket descriptor; otherwise it returns -1.

Introduction to Basic Function Calls

- Binding a socket to a port
 - With servers, the first socket created is often known as a "master socket".
 - A master socket must be bound to a port number so that clients can know where to locate the socket and connect to it.

- Syntax:

```
int bind(int s, struct sockaddr *addr, int addrlen);
```

- Example:

```
struct sockaddr_in address;
```

```
/* type of socket created in socket() */
```

```
address.sin_family = AF_INET;
```

```
address.sin_addr.s_addr = INADDR_ANY;
```

```
/* 7000 is the port to use for connections */
```

```
address.sin_port = htons(7000);
```

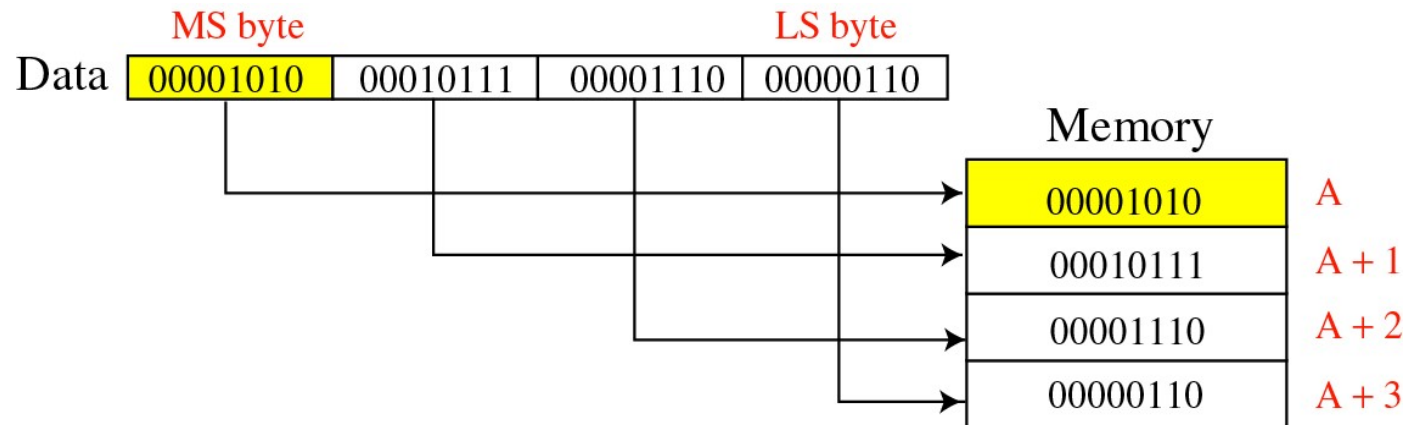
```
/* bind the socket to the port specified above */
```

```
bind(socket_desc, (struct sockaddr *)&address, sizeof(address));
```

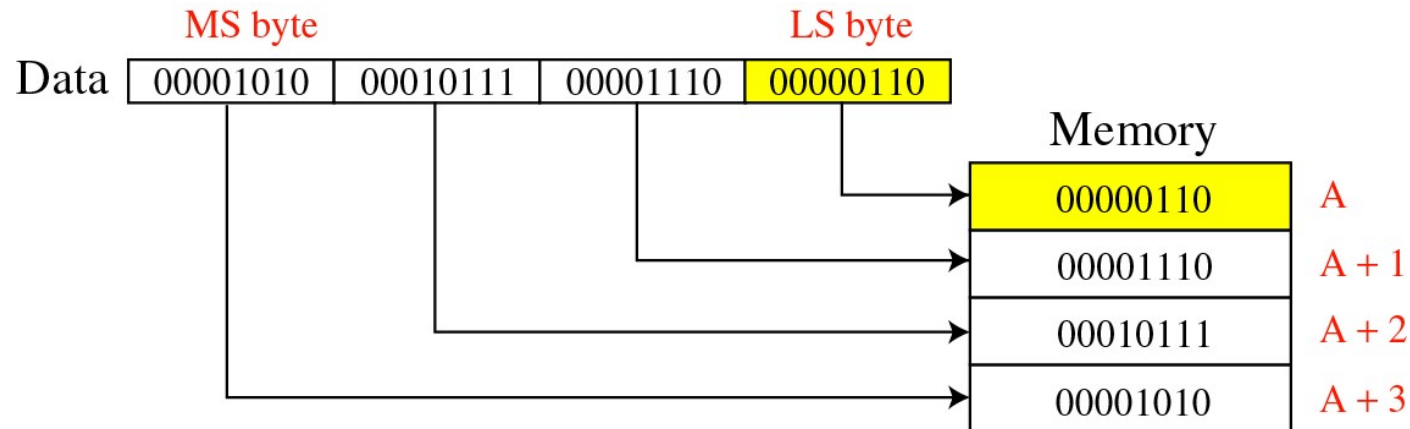
- If successful, bind() returns 0; otherwise it returns -1. The port specified in the source code above (port 7000) is where the server can be connected to.

Byte Ordering

- Big-Endian (network byte):



- Little-Endian (host byte):



Introduction to Basic Function Calls

- Listening for connections
 - Before any connections can be accepted, the socket must be told to listen for connections and also the maximum number of pending connections using `listen()`
- Syntax:
`int listen(int s, int backlog);`
- Example:
`listen(socket_desc,3);`
- The above line specifies that there can be up to 3 connections pending. Function `listen()` applies only to unconnected sockets of type `SOCK_STREAM`.
- If the socket has not been bound to a local port before `listen()` is invoked, the system automatically binds a local port for the socket to listen on.
- If successful, `listen()` returns 0; otherwise it returns -1.

Introduction to Basic Function Calls

- Accepting a connection
 - To tell the server to accept a connection, you have to use the function `accept()`
- Syntax:
`int accept(int s, struct sockaddr *addr, int *addrlen);`
- Example:
`int addrlen;
struct sockaddr_in address;

addrlen = sizeof(struct sockaddr_in);
new_socket = accept(socket_desc, (struct sockaddr *)&address, &addrlen);
if (new_socket < 0)
perror("Accept connection");`
- `accept()` is used with connection based sockets such as streams.
- The parameters are the socket descriptor of the master socket followed by a `sockaddr_in` structure and the size of the structure.
- If successful, it returns a positive integer which is the socket descriptor for the accepted socket. If an error occurs, -1 is returned.

Introduction to Basic Function Calls

- Closing connections
 - A connection can be closed by using `close()`.
- Syntax:
`int close(int sockdes);`
- Example:
`close(socket_desc);`
- `close()` closes the socket descriptor indicated by `sockdes`.
- Upon successful completion, `close()` returns a value of 0; otherwise, it returns -1.

Introduction to Basic Function Calls

- Sending data to a connection
 - Function `send()` is used to transmit a message to another socket and can be used only when the socket is in a connected state.
- Syntax:
`int send(int s, const void *msg, int len, int flags);`
- Example:
`char *message="This is a message to send\n\r";
send(socket_desc, message, strlen(message), 0);`
- The message should have `\n\r` instead of just `\n` or `\r` because otherwise the text which appears on some clients may seem strange.
- The socket descriptor that specifies the socket on which the message will be sent is 's' in the syntax above. 'msg' points to the buffer containing the message and the length of the message is given by len, in bytes.
- Upon successful completion, `send()` returns the number of bytes sent. Otherwise, it returns -1.

Introduction to Basic Function Calls

- Receiving data from a connection
 - Accepting a connection would not be any use without the means to send or receive data. Receive only could be used as a data collection method.
- Syntax:
`int recv(int s, void *msg, int len, int flags);`
- Example:
`int bufsize=1024; /* a 1K buffer */
char *buffer=malloc(bufsize);
recv(socket_desc,buffer,bufsize,0);`
- The flags parameter can be set to MSG_PEEK, MSG_OOB, both, or zero.
- If successful, `recv()` returns the number of bytes received, otherwise, it returns -1. `recv()` returns 0 if the socket is blocking and the connection to the remote node failed.

Introduction to Basic Function Calls

- Setting socket options
 - To allow certain socket operations requires manipulation of socket options using `setsockopt()`

- Syntax:
`int setsockopt(int s, int level, int optname, const void *optval, int optlen);`

- Example:
`#define TRUE 1`
`#define FALSE 0`

```
int socket_desc;    /* master socket returned by socket() */  
int opt=TRUE;      /* option is to be on/TRUE or off/FALSE */
```

```
setsockopt(socket_desc,SOL_SOCKET,SO_REUSEADDR, (char *)&opt,sizeof(opt));
```

- `SOL_SOCKET` specifies the option is a 'socket level' option.
- The option `SO_REUSEADDR` is only valid for `AF_INET` sockets.
- There are two kinds of options: boolean and non-boolean. Boolean options are either set or not set and also can use `optval` and `optlen` to pass information. Non-boolean options always use `optval` and `optlen` to pass information.

Introduction to Basic Function Calls

- Handling more than one connection
 - To enable a socket to be read without waiting

```
#include <process.h>
```

```
i = 100;
```

```
for(i = 0; i < 100; i ++)
```

```
{
```

```
  _beginthread(accept_conn, 0, NULL );
```

```
}//for loop
```

```
_endthread();
```

```
// create a function to accept a connection void accept_conn(void *dummy)
```

```
{
```

```
// doing something here
```

```
}
```

Introduction to Basic Function Calls

- **struct hostent *gethostbyname(const char *name);**

- returns a structure of type hostent for the given host name
- name is a hostname, or an IPv4 address in standard dot notation

e.g. gethostbyname("www.uic.edu.cn") ;

- **struct hostent *gethostbyaddr(const void *addr, socklen_t len, int type);**

- returns a structure of type hostent for the given host address addr of length len and address type type

```
struct hostent {
    char *h_name;           /* official name of host */
    char **h_aliases;       /* alias list (strings) */
    int h_addrtype;         /* host address type (AF_INET) */
    int h_length;           /* length of address */
    char **h_addr_list;     /* list of addresses (binary in network byte order)
                             */
}

#define h_addr h_addr_list[0] /* for backward compatibility */
```

Introduction to Basic Function Calls

- Establishing an outgoing connection
 - To establish a connection to another socket (similar to telnet), use the function `connect()`.

```
int connect(int sockfd, struct sockaddr *serv_addr, int addrlen );
```

- Create the socket using `socket()`, convert the hostname to an IP address using `gethostbyname()` and then issue the `connect()` call passing the relevant structures containing the IP address and port to connect to.

Introduction to Basic Function Calls

```
struct hostent  *he;
struct sockaddr_in server;
int sockfd;

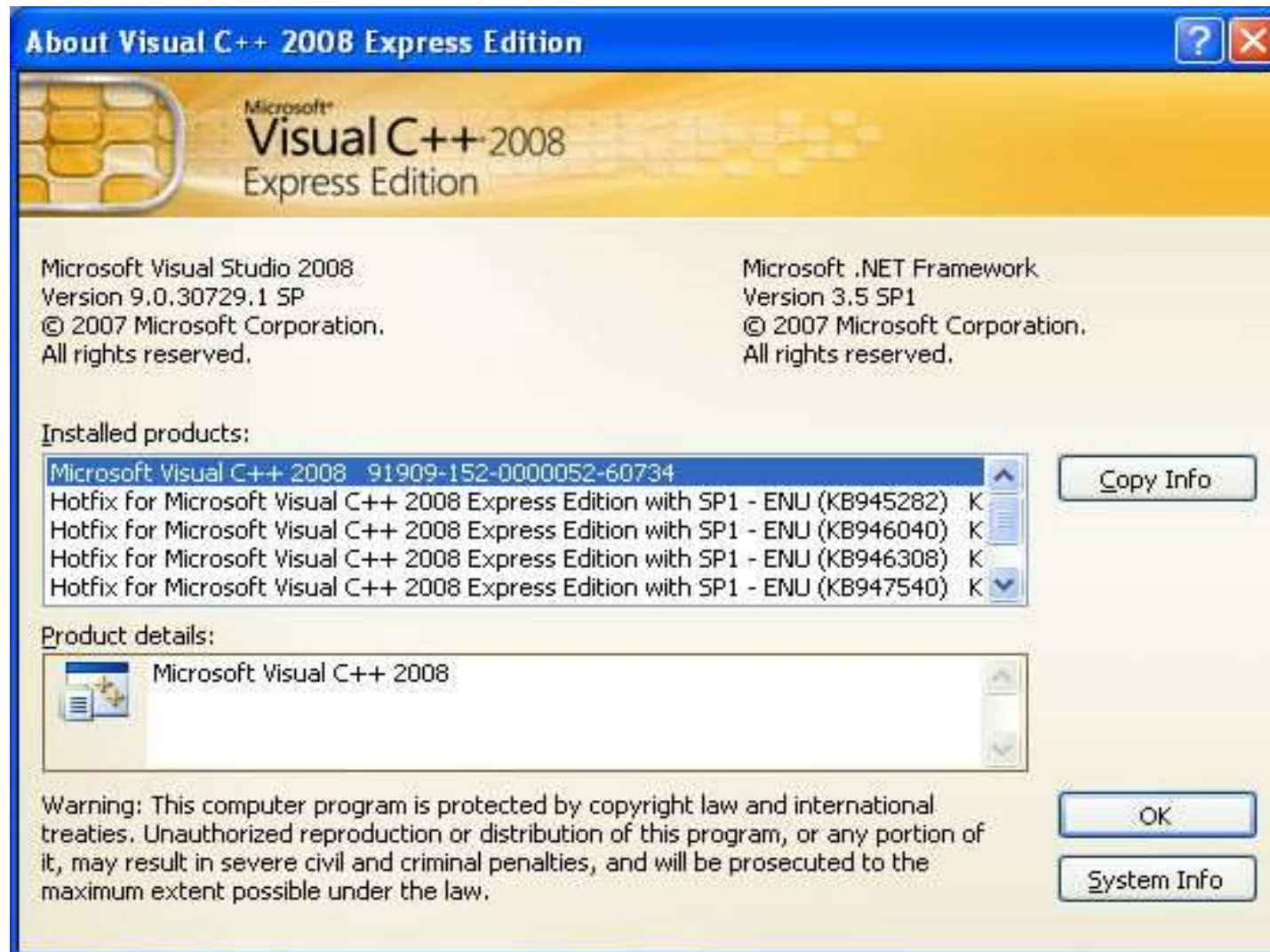
/* resolve localhost to an IP (should be 127.0.0.1) */
if ((he = gethostbyname("localhost")) == NULL)
{ printf("error resolving hostname..");
  exit(1);
}
/* copy the network address part of the structure to the sockaddr_in structure
which is passed to connect() */

memcpy(&server.sin_addr, he->h_addr_list[0], he->h_length);
server.sin_family = AF_INET;
server.sin_port = htons(7000);

/* connect */
if (connect(sockfd, (struct sockaddr *)&server, sizeof(server))
{ printf("error connecting..");
  exit(1);
}
```

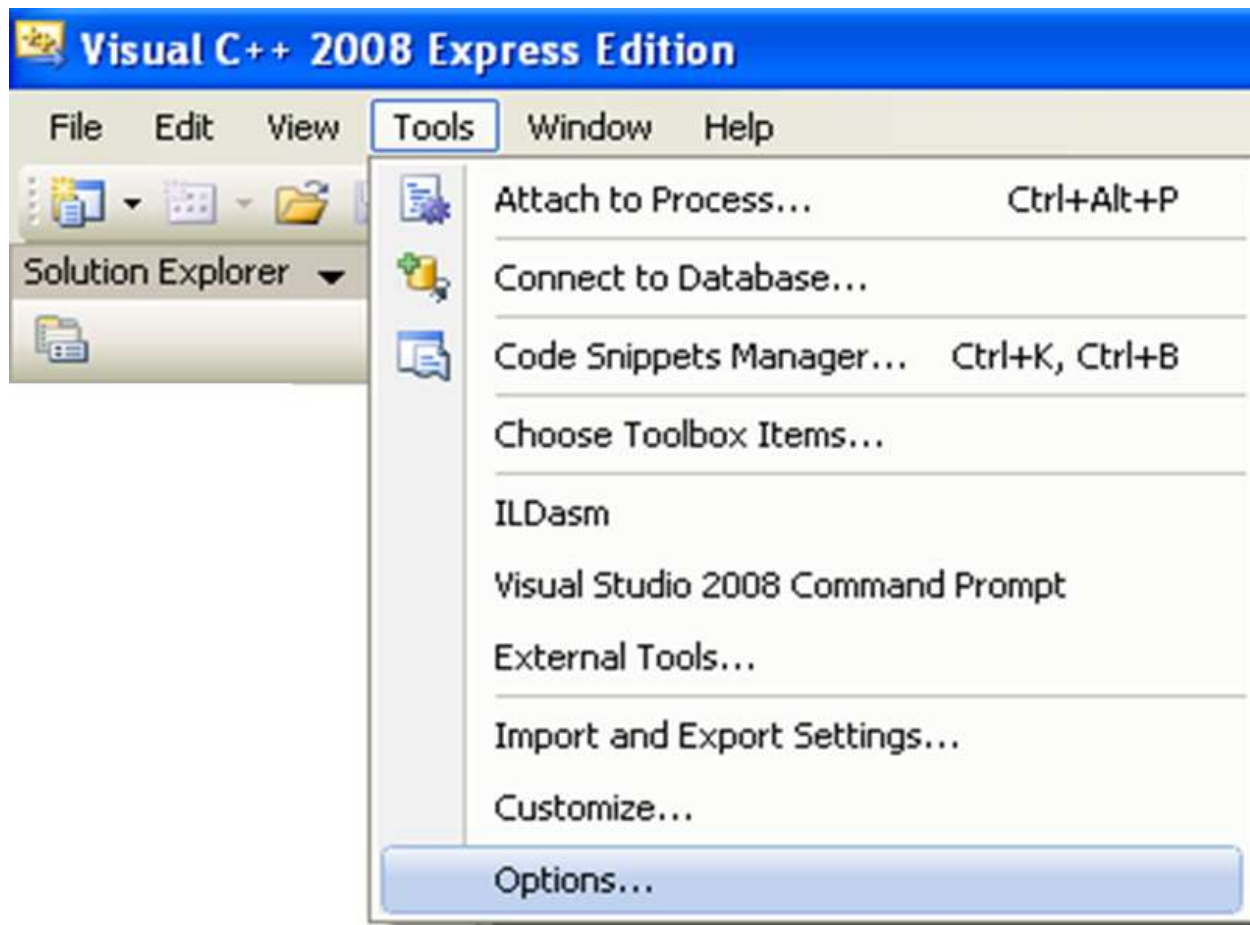
Programming Environment in Windows Platform

- Visual C++ 2008 Express Edition (free version)



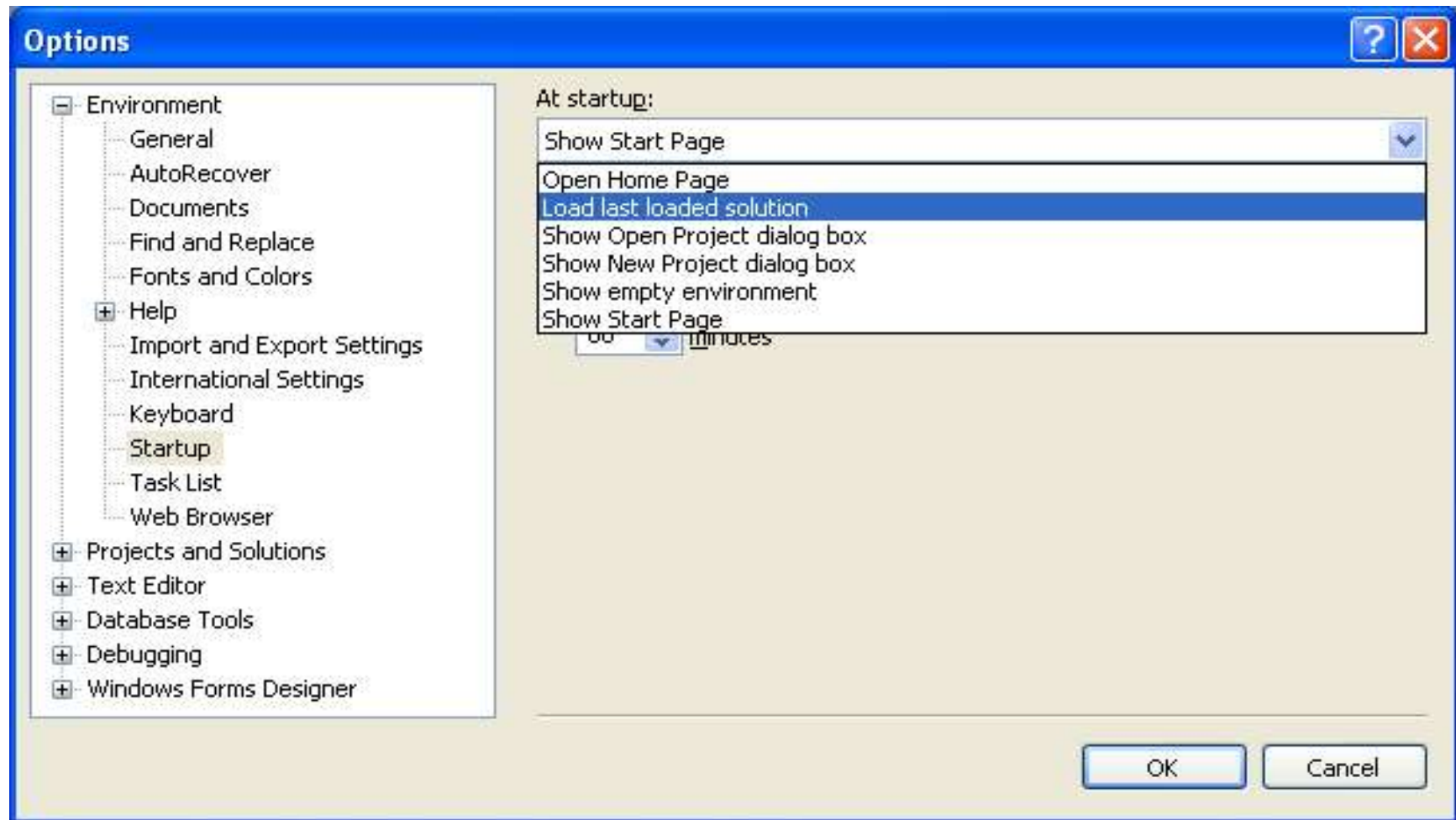
Programming Environment in Windows Platform

- Winsock Header and Libraries
 - Include WINSOCK2.H in your application;
 - Link with WS2_32.LIB library.
 - To compile a program, the steps are as follows: *Click Tools menu > Options sub menu.*



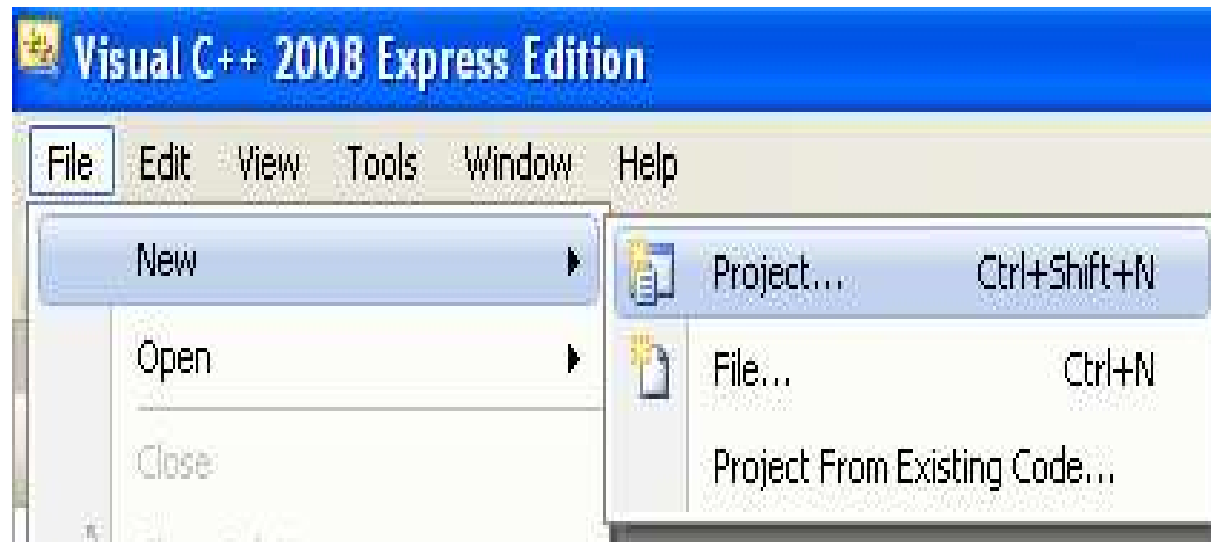
Programming Environment in Windows Platform

- Expand Environment folder > Select Startup link > Set the At Startup: to Load last loaded solution > Click OK.



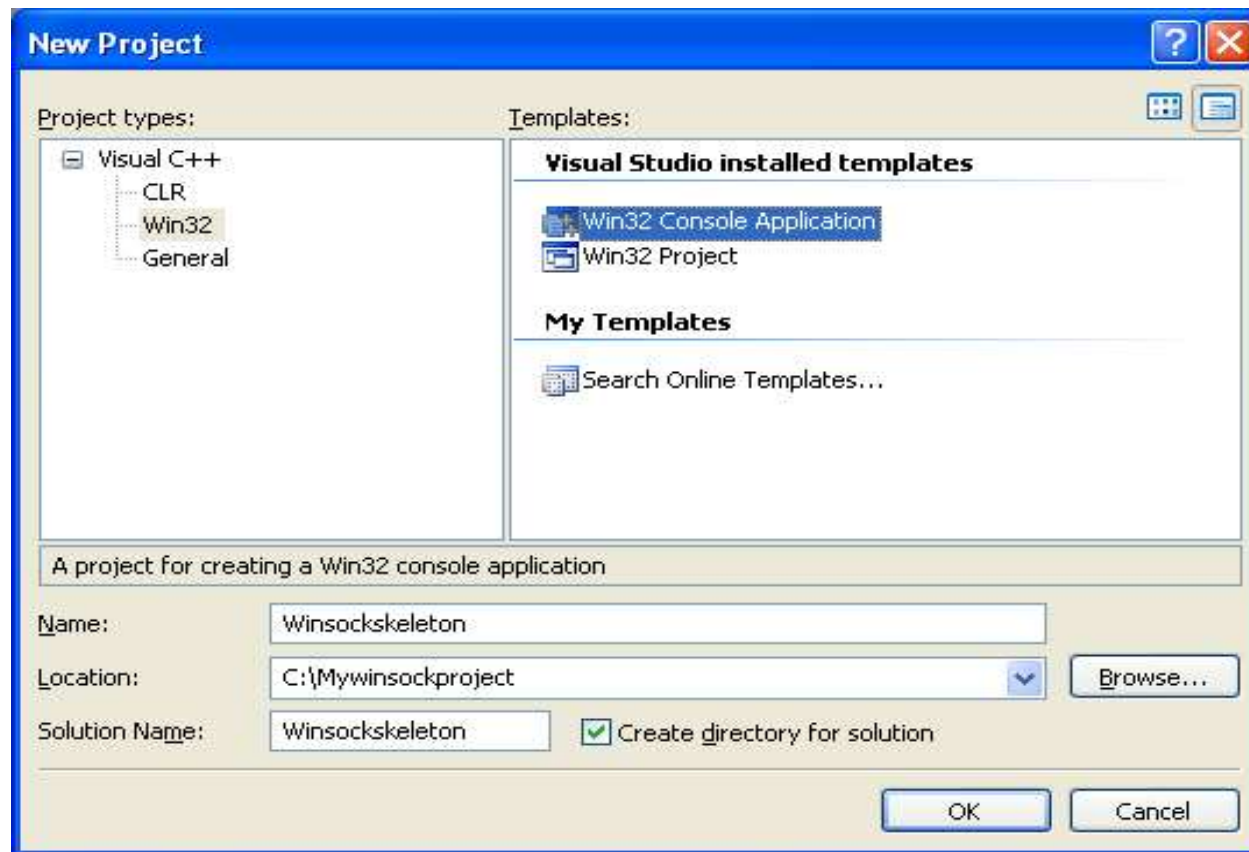
Programming Environment in Windows Platform

- Then we can start creating the Win32 console application project.
- Click File menu > Project sub menu to create a new project.



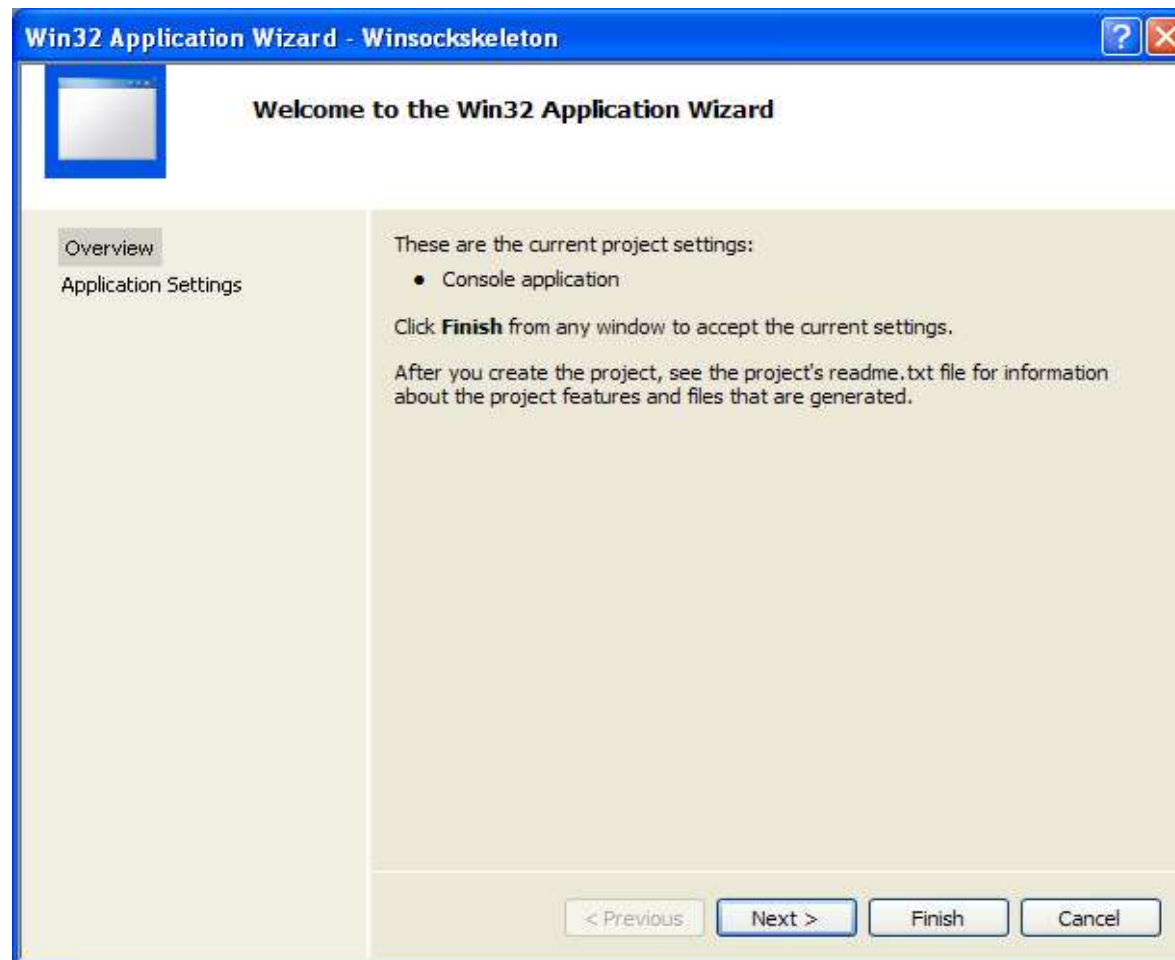
Programming Environment in Windows Platform

- Select Win32 for the Project types and Win32 Console Application for the Templates
- Input the project and solution name.
- Adjust the project location if needed and click OK.



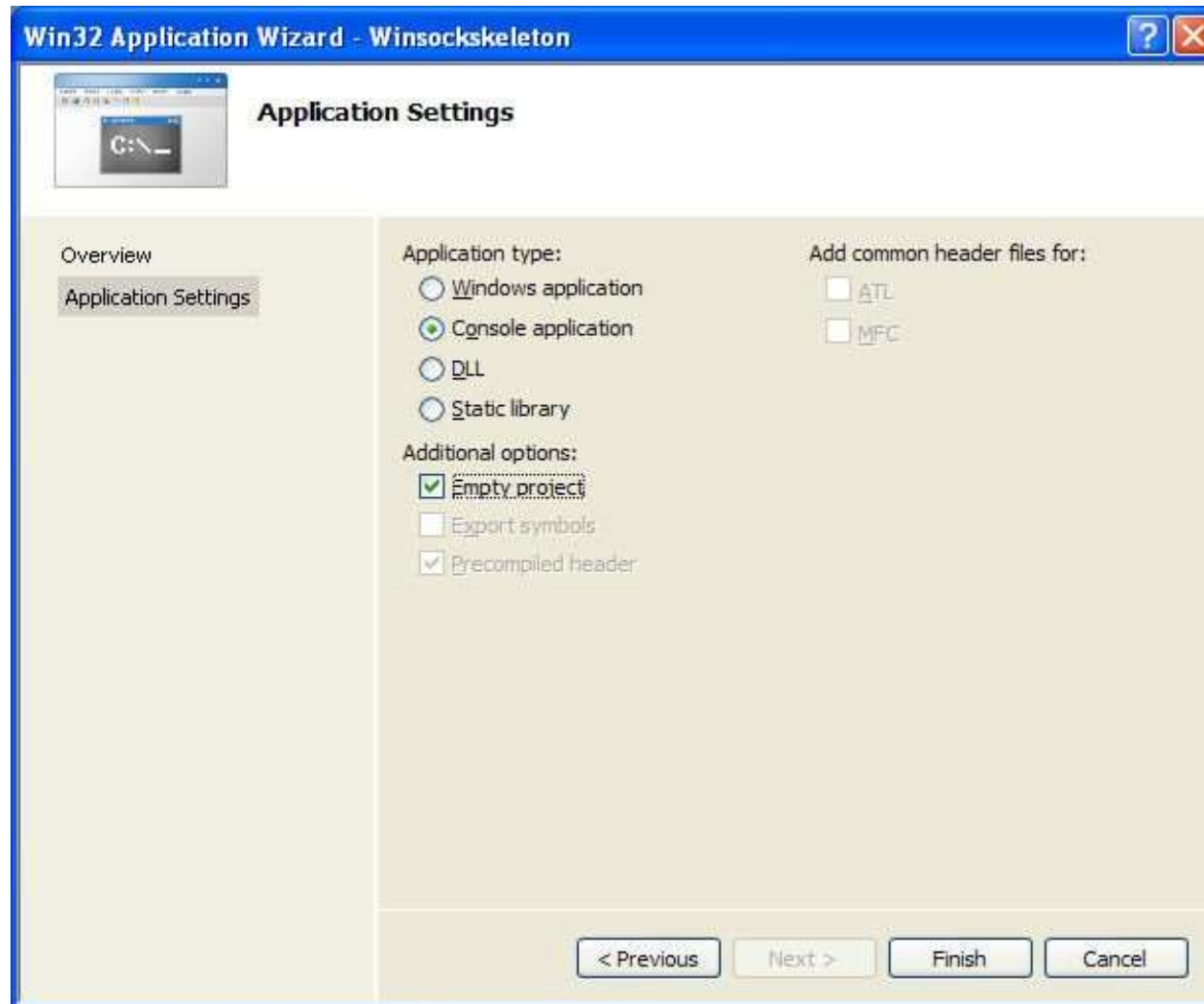
Programming Environment in Windows Platform

- Click Next for the Win32 Application Wizard Overview page.
- We will remove all the unnecessary project items.



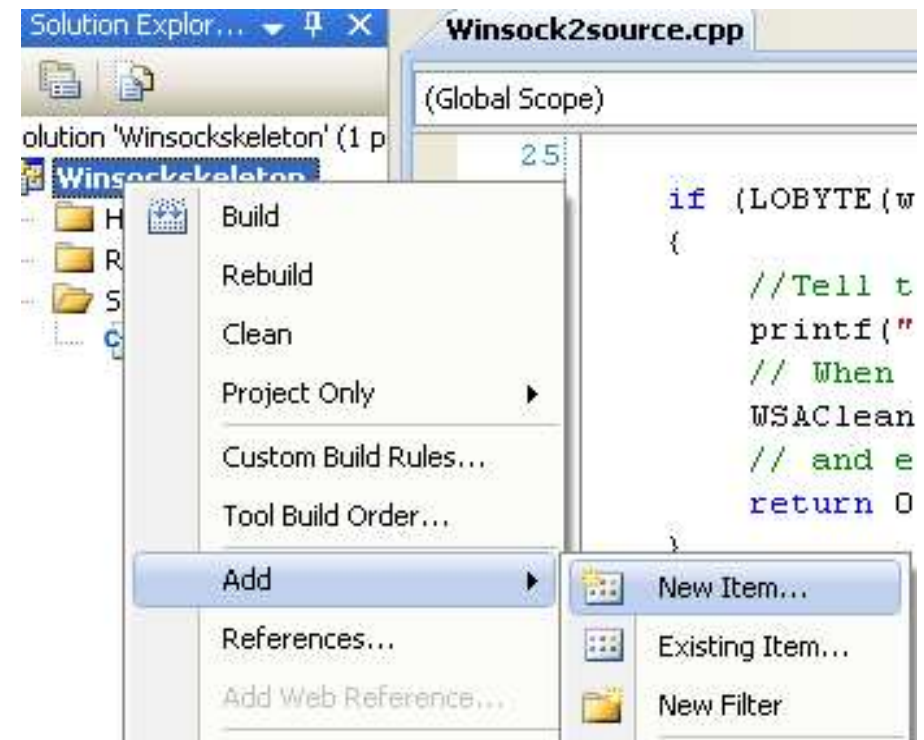
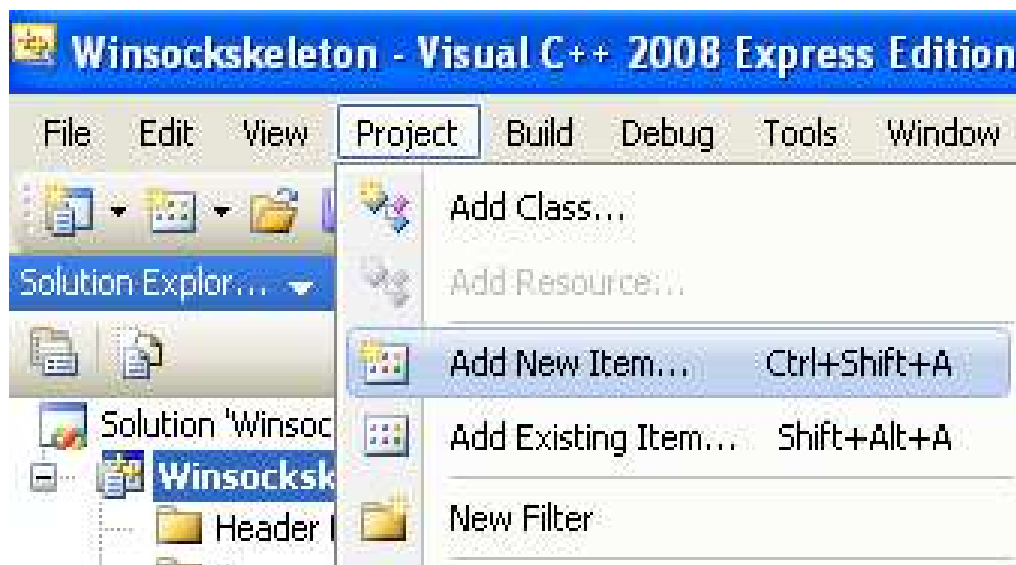
Programming Environment in Windows Platform

- In the Application page, select Empty project for the Additional options.
- Leave others as given and click Finish.



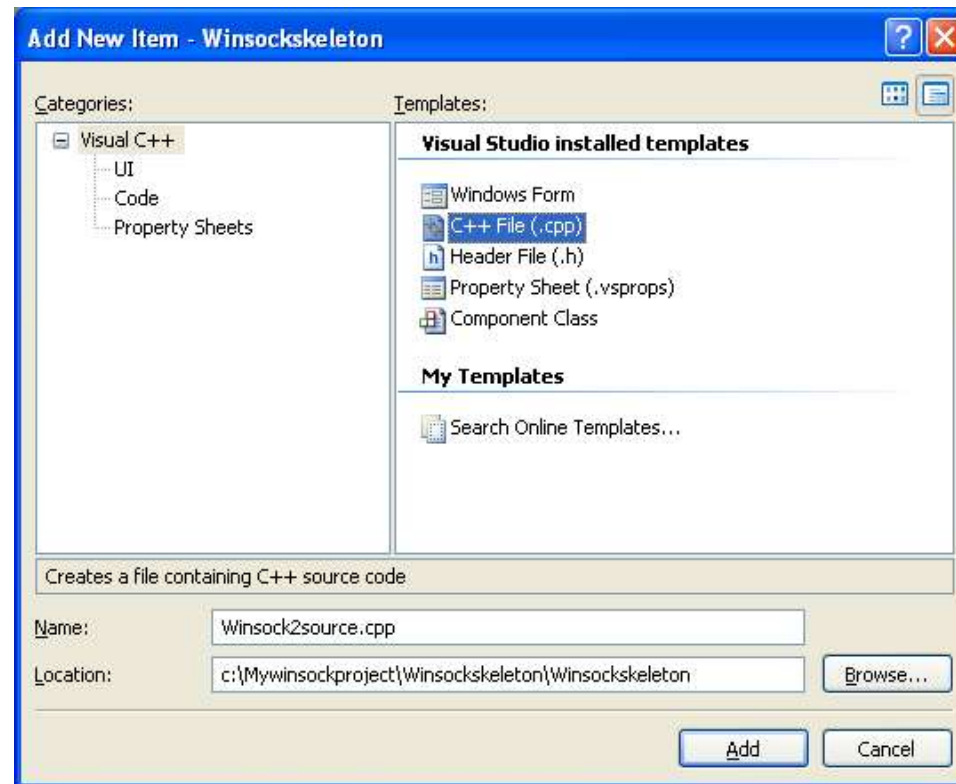
Programming Environment in Windows Platform

- Next, we need to add new source file.
- Click Project menu > Add New Item sub menu or select the project folder in the Solution Explorer > Select Add menu > Select New Item sub menu.



Programming Environment in Windows Platform

- Select C++ File (.cpp) for the Templates: Put the source file name and click Add.
- Although the extension is .cpp, Visual C++ IDE will recognize that the source code used is C based on the Compile as C Code (/TC) option which will be set in the project property page later.



Programming Environment in Windows Platform

- Add the source code ...

```
#include <winsock2.h>
#include <stdio.h>

int main(void)
{
    WSADATA wsaData;
    int RetCode;

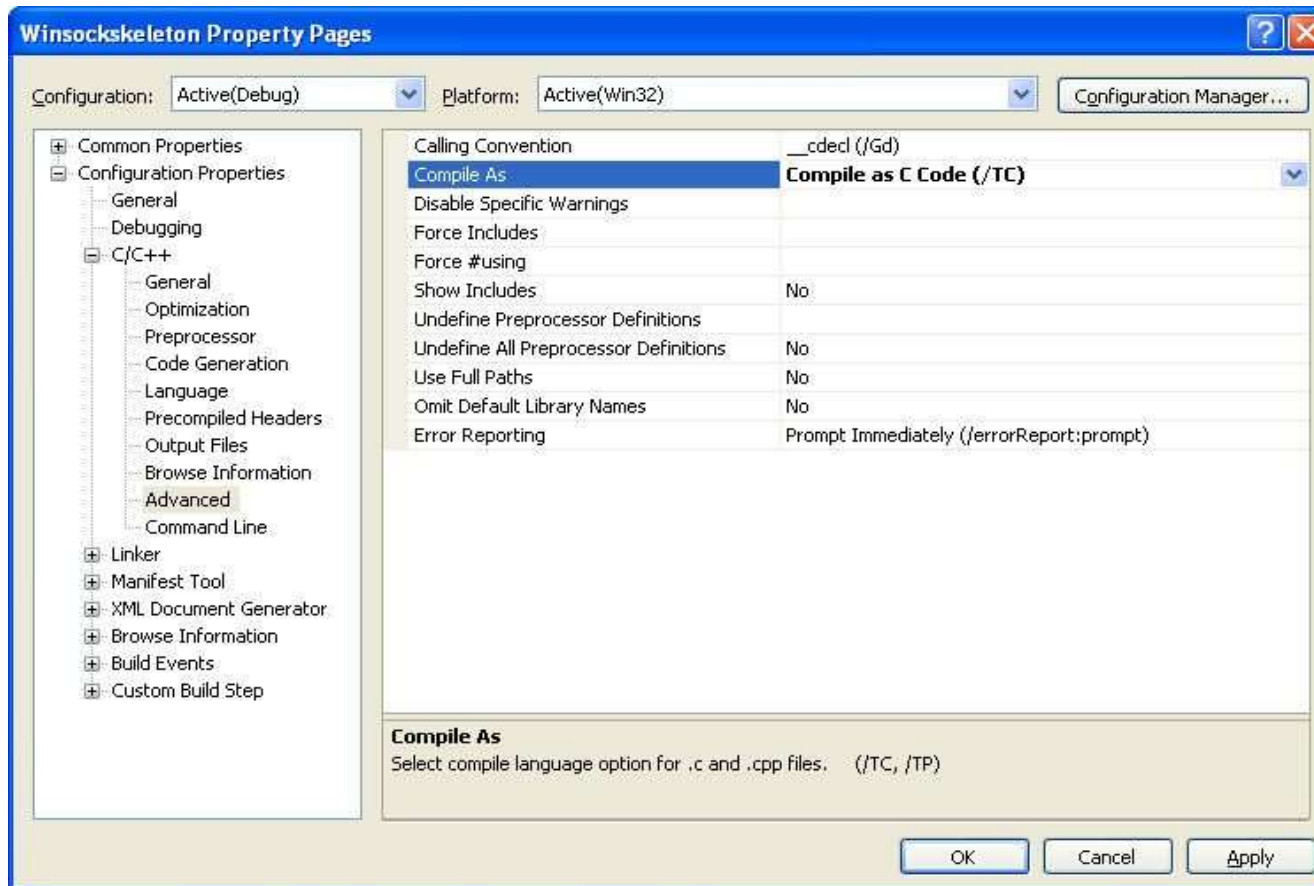
    // Initialize Winsock version 2.2
    if ((RetCode = WSStartup(MAKEWORD(2,2), &wsaData)) != 0)
    {
        printf("WSAStartup failed with error %d\n", RetCode);
        return 1;
    }
}
```

- To build this Winsock C Win32 console application project, we need to set the project to be compiled as C code and link to ws2_32.lib, the Winsock2 library.
- Invoke the project property page.



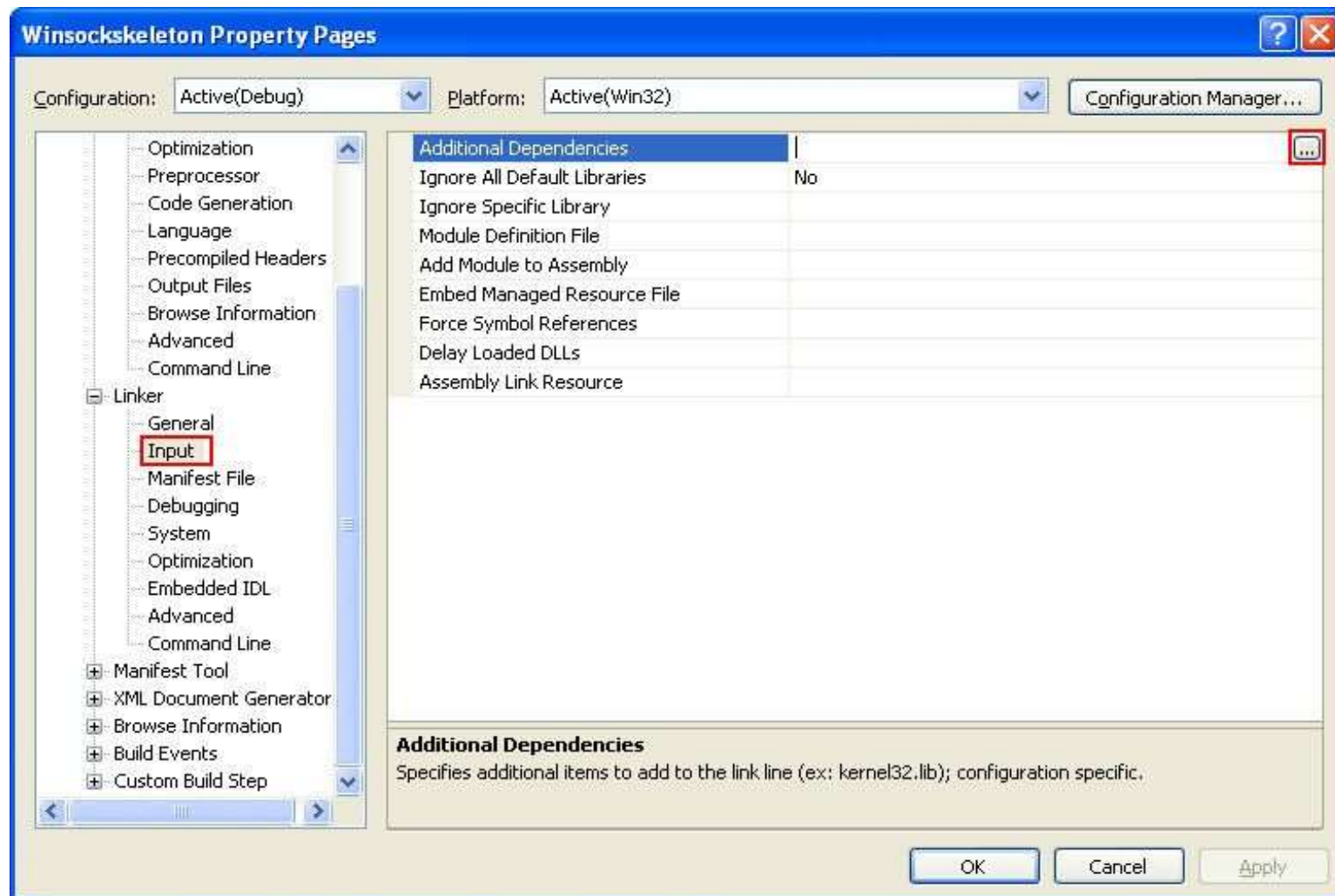
Programming Environment in Windows Platform

- Expand the Configuration folder > Expand the C/C++ sub folder.
- Select the Advanced link and for the Compile As option, select Compile as C Code (/TC).



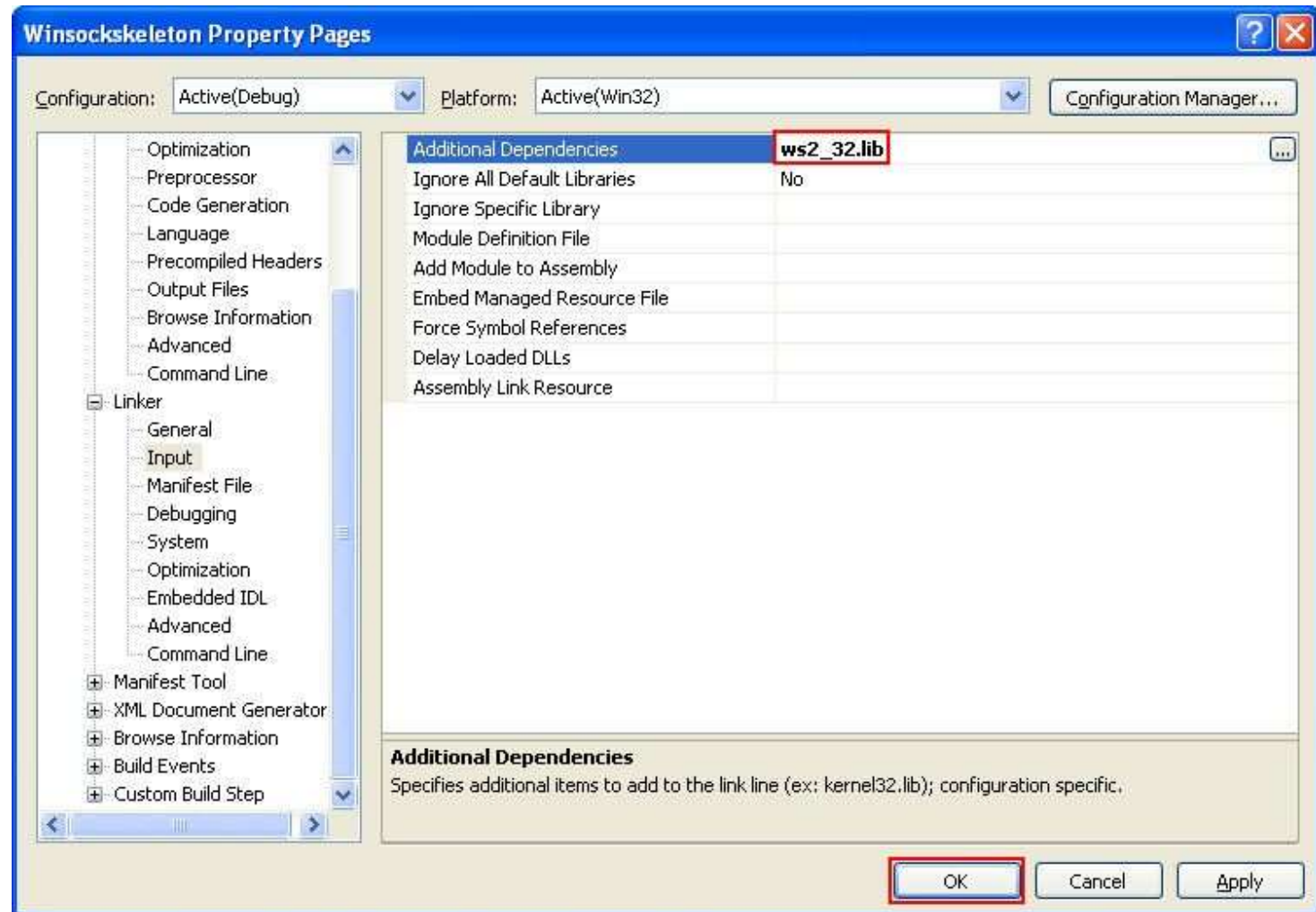
Programming Environment in Windows Platform

- Next, expand the Linker folder and select the Input link.
- For the Additional Dependencies option, click the ellipses at the end of the empty field on the right side.



Programming Environment in Windows Platform

- You can just directly type the library name in the empty field on the right of the Additional Dependencies.
- Click OK.



Programming Environment in Windows Platform

- Build the project and make sure there is no error which can be seen (if any) in the Output window normally docked at the bottom of the IDE by default, and then run the project
- If there is no error, a window will be popped out.

